GuardianTrack

Application de Securité Personnelle

Module	Niveau
Developpement Android Natif avancé (Kotlin)	1ère année Master Professionnel en Développement d'Applications Mobiles
Nature	**Duree estimée**
Mini-Projet Evalué (Individue)	3 à 4 semaines

ENONCE OFFICIEL DU MINI-PROJET

1. Presentation du Projet 3
1.1 Objectifs Pedagogiques 3
2. Specifications Fonctionnelles — Les 4 Piliers 4
2.1 Activity & Fragment 4
2.2 Service 4
Algorithme minimal de detection de chute 4
2.3 BroadcastReceiver 5
2.4 ContentProvider 5
Client de test fourni ..... **Error! Bookmark not defined.**
Securite du ContentProvider 5
3. Persistance des Donnees & Communication 6
3.1 Stockage Local 6
3.2 Stockage Distant & Synchronisation 6
4. Services Natifs & Securite 8
4.1 Geolocalisation 8
4.2 Gestion des Capteurs 8
4.3 Communication d'Urgence 8
4.4 Securite des Donnees 8
5. Contraintes Specifiques Android 12+ (API 31+) 9
6. Architecture & Bonnes Pratiques 10
6.1 Architecture MVVM 10
6.2 Injection de Dependances avec Hilt 10
6.3 Concurrence — Coroutines & Flow 10
6.4 Interface Utilisateur 10
7. Grille d'Evaluation 11
8. Rapport Technique — Questions Obligatoires 12
9. Livrables Attendus 12
10. Ressources & References 13
Documentation officielle 13
Module client de test ..... **Error! Bookmark not defined.**

1. Présentation du Projet

GuardianTrack est une application Android de sécurité personnelle vous permettant de surveiller en temps réel l'état de l'appareil, de stocker des incidents localement et de synchroniser des alertes avec un serveur distant. Ce projet évalue votre maîtrise des composants Android fondamentaux, de l'architecture moderne MVVM, et des bonnes pratiques professionnelles en Kotlin.

1.1 Objectifs Pédagogiques

A l'issue de ce projet, vous devrez être capable de :

- Concevoir une application Android multi-composants en suivant rigoureusement le pattern MVVM.
- Implémenter les quatre composants fondamentaux d'Android : Activity, Service, BroadcastReceiver et ContentProvider.
- Persister des données localement (Room, DataStore, fichiers) et les synchroniser à distance (Retrofit).
- Gérer les permissions dynamiques, la sécurité des données et les contraintes des versions récentes d'Android (API 31+).
- Rédiger un rapport technique rigoureux justifiant vos choix architecturaux.

> Note importante : L'ensemble du code source devra être en Kotlin. L'usage de Java est strictement interdit. L'application doit cibler Android API 26 minimum (Android 8.0) et être compatible avec les dernières versions.

2. Spécifications Fonctionnelles — Les 4 Piliers

L'application doit impérativement implémenter les quatre composants majeurs du système Android. Chaque pilier est évalué indépendamment selon la grille d'évaluation fournie en Section 7.

2.1 Activity & Fragment

Architecture requise : Single Activity avec Jetpack Navigation Component.

L'application comporte une MainActivity unique accompagnée de trois Fragments principaux :

Fragment	Nom	Responsabilites
Fragment 1	**DashboardFragment**	Affiche en temps réel : état des capteurs, niveau batterie, statut GPS, bouton d'alerte manuelle.
Fragment 2	**HistoryFragment**	Liste paginée des incidents (RecyclerView + DiffUtil). Accàs à l'export CSV/TXT. Suppression par swipe.
Fragment 3	**SettingsFragment**	Paramètres utilisateur : seuil de sensibilité, mode sombre, numéro d'urgence, mode simulation SMS.

> Critère d'évaluation additionnel : Un incident sera simulé par l'évaluateur lors d'une rotation d'écran. Votre ViewModel doit survivre au changement de configuration sans perte de données ni crash.

2.2 Service

Type requis : Foreground Service avec notification persistante.

Le service SurveillanceService tourne en arrière-plan et écoute en permanence les données de l'accéléromètre. Il doit implémenter l'algorithme de détection de chute ci-dessous.

Algorithme minimal de detection de chute

L'algorithme se décompose en deux phases temporelles consécutives observées sur l'axe Z :

Phase	Critere	Description
1 — Free-fall	Magnitude < 3 m/s2 pendant > 100 ms	L'appareil est en chute libre : la somme vectorielle des accelerations est quasi nulle.
2 — Impact	Magnitude > 15 m/s2 dans une fenetre de 200 ms	L'appareil heurte une surface : pic brusque d'acceleration apres la phase de chute.

Formule de la magnitude : **magnitude = sqrt(ax^2 + ay^2 + az^2)**
Le seuil de détection (15 m/s2) doit être paramétrable depuis le SettingsFragment. Des améliorations algorithmiques (filtres, machine learning) seront valorisées en bonus.

2.3 BroadcastReceiver

Vous devez implémenter et enregistrer les receivers suivants :

Action	Comportement attendu	Remarques
ACTION_BATTERY_LOW	Enregistre un incident de type BATTERIE_CRITIQUE dans Room, puis envoie une notification locale de dernier recours.	Enregistrer en manifest (statique).
ACTION_BOOT_COMPLETED	Relance le SurveillanceService automatiquement apres un redemarrage du telephone.	Voir contraintes Android 12+ en Section 5.

Contrainte Android 12+ (API 31) : Depuis Android 12, les applications ne peuvent plus démarrer des services en arrière-plan directement depuis un BroadcastReceiver sauf exceptions spécifiques. Vous devez utiliser WorkManager avec setExpedited(OutOfQuotaPolicy.RUN_AS_NON_EXPEDITED_WORK_REQUEST) comme solution de repli. Justifiez votre stratégie de compatibilité dans votre rapport technique.

2.4 ContentProvider

Vous devez créer un ContentProvider personnalisé exposant la liste des contacts d'urgence enregistrés dans votre base de données Room.

URI contractuelle imposée : content://com.guardian.track.provider/emergency_contacts

Le ContentProvider doit exposer au minimum les colonnes suivantes :

Colonne	Type	Description
_id	INTEGER	Identifiant unique auto-incremente.
name	TEXT	Nom du contact d'urgence.
phone_number	TEXT	Numero de téléphone (chiffre uniquement).

Sécurité du ContentProvider

Vous devez protéger votre ContentProvider avec une permission personnalisée dans le Manifest :

<permission android:name="com.guardian.track.READ_EMERGENCY_CONTACTS" android:protectionLevel="signature|privileged" /> Justifiez dans votre rapport technique le choix du niveau de protection (normal, dangerous, signature, signature|privileged) et ses implications en termes de sécurité inter-applications.

3. Persistance des Données & Communication

3.1 Stockage Local

Room Database — Stockage principal des incidents.
La base de données Room doit contenir au minimum les entités et DAO suivants :

Entite / DAO	Champs / Methodes requises
IncidentEntity	id (PK), timestamp (Long), type (String : FALL | BATTERY | MANUAL), latitude (Double), longitude (Double), isSynced (Boolean)
EmergencyContactEntity	id (PK), name (String), phoneNumber (String)
IncidentDao	insertIncident(), getAllIncidents() : Flow<List<IncidentEntity>>, getUnsyncedIncidents(), markAsSynced(id)
EmergencyContactDao	insertContact(), deleteContact(), getAllContacts() : Flow<List<EmergencyContactEntity>>

DataStore (Preferences) — Stockage des preferences utilisateur.

- Seuil de sensibilité de l'accéléromètre (Float, default: 15.0 m/s2)
- Mode sombre actif (Boolean)
- Numéro d'urgence (String chiffré)
- Mode simulation SMS actif (Boolean)

File Storage (Scoped Storage) — Export de l'historique.
L'export doit produire un fichier .csv ou .txt dans le dossier public Documents/ via MediaStore ou SAF (Storage Access Framework). Le fichier doit contenir : Date, Heure, Type d'incident, Latitude, Longitude, Statut de synchronisation.

3.2 Stockage Distant & Synchronisation

Vous utiliserez Retrofit pour l'envoi des alertes. La stratégie de synchronisation doit suivre le schéma offline-first :

Scenario	Comportement requis
Reseau disponible	Envoi immédiat via Retrofit. Si succes : marquer isSynced = true dans Room.
Pas de reseau	Stocker l'incident dans Room (isSynced = false). Planifier une tâche WorkManager avec Constraints (NETWORK_CONNECTED) pour la synchronisation différée.
Retour du reseau	WorkManager reprend automatiquement et synchronise tous les incidents en attente (isSynced = false).

Les états de chargement et d'erreur réseau doivent être modélisés avec des Sealed Classes Kotlin :

```
sealed class NetworkResult<T> {    data class Success<T>(val data: T) : NetworkResult<T>()    data class Error<T>(val message: String, val code: Int? = null) : NetworkResult<T>()    class Loading<T> : NetworkResult<T>() }
```

API cible : Vous pouvez utiliser MockAPI (mockapi.io), un serveur Firebase Realtime Database, ou tout autre backend REST acceptant du JSON. L'URL de base de l'API doit être configurable via un fichier local.properties non versionné (ajouter au .gitignore).

4. Services Natifs & Sécurité

4.1 Géolocalisation

Utilisez FusedLocationProviderClient pour récupérer les coordonnées GPS lors de chaque incident. Intégrez les coordonnées dans le log Room et dans la requête envoyée à l'API distante.
Cas de repli : Si la permission GPS est refusée définitivement (shouldShowRequestPermissionRationale retourne false), affichez une dialog expliquant comment activer la permission dans les paramètres système, et enregistrez l'incident avec latitude = 0.0 / longitude = 0.0 (valeur sentinelle). Justifiez cette stratégie dans votre rapport.

4.2 Gestion des Capteurs

Utilisez SensorManager pour écouter l'accéléromètre (TYPE_ACCELEROMETER) dans votre SurveillanceService. Les callbacks du SensorEventListener doivent être traités sur un thread dédié (HandlerThread ou Coroutine Dispatcher.Default) pour éviter de bloquer le thread principal.

4.3 Communication d'Urgence

En cas d'alerte critique (chute détectée ou batterie critique), l'application doit tenter d'envoyer un SMS automatique au numéro d'urgence configuré dans les paramètres.

> OBLIGATOIRE — Mode Simulation SMS : Un toggle 'Mode Simulation' doit etre accessible dans le SettingsFragment. Lorsqu'il est activé (valeur par défaut : ACTIVE), les SMS sont remplacés par une notification locale et un log console. Ce mode est obligatoire pour les tests en environnement académique et doit être la valeur par défaut de l'application.

Lorsque le mode réel est activé, l'envoi de SMS doit utiliser SmsManager.sendTextMessage(). Le numéro de destination doit être celui stocké dans DataStore. Tout envoi de SMS à un numéro non prévu est de la seule responsabilité de l'étudiant.

4.4 Sécurité des Données

- EncryptedSharedPreferences (Jetpack Security) : Chiffrer le numéro d'urgence et la clé d'API stockés localement.
- Permissions dynamiques : Demander au runtime les permissions ACCESS_FINE_LOCATION, SEND_SMS, et POST_NOTIFICATIONS (Android 13+). Afficher une explication pédagogique avant chaque demande.
- ContentProvider : Protéger l'accès par la permission personnalisée définie en Section 2.4.
- Fichier local.properties : Ne pas versionner la clé API. Documenter ce point dans le README.

5. Contraintes Spécifiques Android 12+ (API 31+)

Section obligatoire : Les points ci-dessous doivent être traités explicitement dans votre code ET dans votre rapport technique. Ignorer ces contraintes entrainera des points de malus.

Contrainte	Solution attendue
Démarrage de service depuis BroadcastReceiver (BOOT_COMPLETED)	Utiliser WorkManager avec setExpedited() comme solution de repli sur API 31+. Tester et documenter le comportement observe.
Foreground Service Type (Android 14, API 34)	Déclarer android:foregroundServiceType="location|microphone" selon les capteurs utilises dans le Manifest.
Permission POST_NOTIFICATIONS (Android 13, API 33)	Demander la permission dynamiquement avant d'afficher toute notification. Gérer le refus de l'utilisateur sans crash.
Exact Alarms — SCHEDULE_EXACT_ALARM (Android 12)	Si des alarmes exactes sont utilisees, demander la permission USE_EXACT_ALARM ou SCHEDULE_EXACT_ALARM et gerer le cas ou elle est refusee.

6. Architecture & Bonnes Pratiques

6.1 Architecture MVVM

L'architecture MVVM est obligatoire et sera évaluée rigoureusement. La séparation des responsabilités doit être stricte :

Couche	Classes / Composants	Responsabilite
View	Fragments, Activities	Affichage uniquement. Aucune logique métier. Observe les StateFlow/LiveData du ViewModel.
ViewModel	XxxViewModel (HiltViewModel)	Logique de presentation. Survit aux rotations. Expose des StateFlow<UiState> immutables.
Repository	IncidentRepository, ContactRepository	Source de vérité unique. Combine Room + Retrofit + WorkManager. Injecté via Hilt.
Model	Entity, DTO, DomainModel	Données pures. Séparation Entity (Room) / DTO (Retrofit) / DomainModel (UI).

6.2 Injection de Dépendances avec Hilt

Hilt est obligatoire. Vous devez fournir via des modules Hilt les instances suivantes :

- L'instance Room Database (singleton)
- Le client Retrofit (singleton, avec intercepteur pour la cle API)
- Les instances DAO et Repository
- FusedLocationProviderClient

6.3 Concurrence — Coroutines & Flow

Toute opération asynchrone doit utiliser les Coroutines Kotlin. L'usage de Thread, AsyncTask ou RxJava est interdit.

- Room : retourner des Flow<List<...>> depuis les DAO, collectes dans les ViewModels via viewModelScope.
- Retrofit : suspending functions dans les interfaces de service.
- Capteurs : traitement sur Dispatchers.Default, mise a jour UI via StateFlow sur Dispatchers.Main.

6.4 Interface Utilisateur

L'interface doit respecter les principes de Material Design 3. ViewBinding est obligatoire (Jetpack Compose est accepté en remplacement complet avec bonus de 5 points). L'application doit supporter le mode sombre (DayNight theme).

7. Grille d'Evaluation

La note finale est sur 100 points.

Critere d'Evaluation	Detail	Points
Architecture MVVM + Hilt	Separation stricte View/ViewModel/Repository, modules Hilt, survie a la rotation	**20**
Foreground Service + Detection Capteur	Notification persistante, algorithme free-fall/impact implemente, parametrable	**20**
Room + DataStore + Export Fichier	Entites et DAO corrects, preferences persistees, export CSV/TXT fonctionnel	**15**
Retrofit + Sealed Classes + WorkManager	Envoi d'alerte, gestion offline-first, synchronisation differee WorkManager	**10**
BroadcastReceiver (BOOT + BATTERY)	Les deux receivers implementes et fonctionnels, compatibilite Android 12+ justifiee	**10**
ContentProvider sécurisé	URI contractuelle respectée, testée avec le module :app-client, permission signature déclarée	**10**
Permissions Dynamiques + UX	Rationale affiche, cas DENIED_PERMANENTLY gere, mode simulation SMS actif par defaut	**10**
Rapport Technique	Reponses contextualisees aux questions obligatoires (cf. Section 8)	**5**
TOTAL		**100**
BONUS — Jetpack Compose (remplacement complet de ViewBinding)	UI entièrement implémentée en Compose avec animations Material 3	**+5**
BONUS — Algorithme de détection amélioré	Filtre passe-bas, fenêtre glissante, machine learning embarque (TFLite)	**+3**
BONUS — Tests unitaires (ViewModel, Repository)	Au moins 10 tests unitaires significatifs avec JUnit4 + MockK ou Turbine	**+2**

8. Rapport Technique — Questions Obligatoires

Le rapport technique (format PDF, 5 a 10 pages) doit répondre de maniére contextualisée et argumentée aux questions suivantes. Des réponses génériques copiées de la documentation officielle ne seront pas acceptées.

N°	Question
1	Pourquoi avoir choisi Flow plutôt que LiveData pour la communication entre Room et le ViewModel ? Dans quel cas auriez-vous opté pour LiveData dans votre projet spécifiquement ?
2	Comment avez-vous géré le cas où la permission ACCESS_FINE_LOCATION est refusée définitivement par l'utilisateur (shouldShowRequestPermissionRationale retourne false) ? Quelle est votre stratégie de repli et pourquoi ?
3	Quelles sont les limites de sécurité de votre ContentProvider ? Qu'est-ce qu'une attaque de type 'content provider injection' et comment vous en protégez-vous ?
4	Décrivez les restrictions de démarrage de services en arrière-plan introduites en Android 12 (API 31). Comment votre implémentation de ACTION_BOOT_COMPLETED contourne-t-elle ces restrictions de manière propre ?
5	Pourquoi séparez-vous les entités Room (Entity), les objets réseau (DTO) et les modèles de domaine (DomainModel) ? Montrez un exemple concret tiré de votre code ou cette séparation apporte une valeur ajoutée.
6	Quelle est la différence entre WorkManager, JobScheduler et AlarmManager ? Justifiez le choix de WorkManager pour la synchronisation différée dans le contexte de GuardianTrack.

9. Livrables Attendus

N°	Livrable	Exigences
1	**Code Source**	Depôt GitHub public (ou privé avec accès partagé). Commits réguliers avec messages clairs. Fichier README.md documentant : configuration de l'API, modules requis, instructions de build. Fichier .gitignore excluant local.properties.
2	**Rapport Technique (PDF)**	5 a 10 pages. Réponses aux 6 questions de la Section 8. Diagramme d'architecture MVVM de votre application. Schema de la base de données Room.
3	**Video de démonstration**	Durèe : 3 à 5 minutes. Scenario obligatoire a filmer : (1) Simulation d'une chute via les capteurs, (2) Envoi de la notification SMS simulée, (3) Enregistrement dans Room visible dans l'historique, (4) Export CSV du log.

Délai de rendu : Samedi 18 Avril 2026. Tout retard non justifie entraine une pénalité de 2 points par jour calendaire de retard, dans la limite de 10 points.

10. Ressources & Références

Documentation officielle

- Android Developers — Guide to App Architecture : developer.android.com/topic/architecture
- Hilt Dependency Injection : developer.android.com/training/dependency-injection/hilt-android
- Room Persistence Library : developer.android.com/training/data-storage/room
- WorkManager Guide : developer.android.com/topic/libraries/architecture/workmanager
- Foreground Services : developer.android.com/guide/components/foreground-services
- Background restrictions (Android 12+) : developer.android.com/about/versions/12/behavior-changes-12

> Bonne chance à tous. Ce projet est conçu pour vous confronter aux défis réels du développement Android professionnel. La qualité du code, la rigueur architecturale et la clarté du rapport comptent autant que le nombre de fonctionnalités implémentées.